

Python Coding Practice: Lists, Dictionaries, List Comprehensions, Strings & Arrays

30 interview-style problems with complete solutions, explanations, edge cases, and complexity notes.

How to practice: Read only the question first. Give yourself 15–25 minutes. If stuck, study the approach, hide the solution, and rewrite it yourself. The goal is pattern recognition—not memorizing code.

LISTS

1. Find the largest element without max()

Question: Given a list of integers, return the largest value without using max().

Solution:

```
def find_largest(nums):
    if not nums:
        return None
    largest = nums[0]
    for num in nums[1:]:
        if num > largest:
            largest = num
    return largest

print(find_largest([4, 9, 2, 15, 6]))
# 15
```

Explanation: Keep one variable as the best value seen so far. Scan once and replace it whenever a larger value appears.

Complexity: O(n) time, O(1) extra space.

2. Remove duplicates while preserving order

Question: Remove duplicate values while keeping the first occurrence of each value.

Solution:

```
def remove_duplicates(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_duplicates([3, 1, 3, 2, 1, 4]))
# [3, 1, 2, 4]
```

Explanation: A set gives fast membership checking, while the result list preserves input order.

Complexity: O(n) average time, O(n) space.

3. Find the second largest distinct number

Question: Return the second largest distinct value. Return None if it does not exist.

Solution:

```
def second_largest(nums):
    first = second = None

    for num in nums:
        if num == first or num == second:
            continue
        if first is None or num > first:
            second = first
            first = num
        elif second is None or num > second:
```

```

second = num

return second

print(second_largest([10, 5, 10, 8, 3]))
# 8

```

Explanation: Track the largest and second-largest distinct values as you scan. No sorting is required.

Complexity: $O(n)$ time, $O(1)$ extra space.

4. Move all zeros to the end

Question: Move zeros to the end while preserving the order of non-zero elements.

Solution:

```

def move_zeros(nums):
    write = 0
    for num in nums:
        if num != 0:
            nums[write] = num
            write += 1
    while write < len(nums):
        nums[write] = 0
        write += 1
    return nums

print(move_zeros([0, 1, 0, 3, 12]))
# [1, 3, 12, 0, 0]

```

Explanation: The write pointer records where the next non-zero value belongs. The remaining positions are filled with zero.

Complexity: $O(n)$ time, $O(1)$ extra space.

5. Find the missing number

Question: The list contains distinct numbers from 0 through n with one missing. Find it.

Solution:

```

def missing_number(nums):
    n = len(nums)
    expected = n * (n + 1) // 2
    return expected - sum(nums)

print(missing_number([3, 0, 1]))
# 2

```

Explanation: Compare the mathematical sum of $0..n$ with the actual sum.

Complexity: $O(n)$ time, $O(1)$ extra space.

6. Rotate a list right by k

Question: Rotate [1,2,3,4,5] by 2 to get [4,5,1,2,3].

Solution:

```
def rotate(nums, k):
    if not nums:
        return nums
    k %= len(nums)
    nums[:] = nums[-k:] + nums[:-k]
    return nums

print(rotate([1, 2, 3, 4, 5], 2))
# [4, 5, 1, 2, 3]
```

Explanation: Modulo handles k larger than the list length. Slicing makes the operation simple and readable.

Complexity: O(n) time, O(n) temporary space.

DICTIONARIES / HASHING

7. Count frequency of each element

Question: Return a dictionary mapping every value to its frequency.

Solution:

```
def frequency_count(items):
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1
    return freq

print(frequency_count(["a", "b", "a", "c", "b", "a"]))
# {'a': 3, 'b': 2, 'c': 1}
```

Explanation: dict.get(key, 0) is one of the most useful Python patterns for counting.

Complexity: O(n) time, O(k) space for k distinct values.

8. First non-repeating character

Question: Return the first character occurring exactly once, or None.

Solution:

```
def first_unique_char(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1

    for ch in s:
        if freq[ch] == 1:
            return ch
    return None

print(first_unique_char("swiss"))
# w
```

Explanation: Count first, then scan again. The second scan preserves the original order.

Complexity: O(n) time, O(k) space.

9. Two Sum

Question: Return indices of two numbers adding to target.

Solution:

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

print(two_sum([2, 7, 11, 15], 9))
# [0, 1]
```

Explanation: Instead of checking every pair, store previous values in a dictionary and look for the required complement.

Complexity: O(n) average time, O(n) space.

10. Group anagrams

Question: Group words containing the same letters.

Solution:

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        key = tuple(sorted(word))
        groups[key].append(word)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Explanation: Anagrams have identical sorted characters, so the sorted word can be used as a dictionary key.

Complexity: O(n * m log m), where m is average word length.

11. Most frequent element

Question: Return the most frequent element; if tied, return the one appearing first.

Solution:

```
def most_frequent(items):
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1

    best = None
    best_count = 0
    for item in items:
        if freq[item] > best_count:
            best = item
            best_count = freq[item]
    return best

print(most_frequent([3, 1, 3, 2, 1, 3, 2]))
# 3
```

Explanation: One pass counts; another pass preserves input order when frequencies tie.

Complexity: O(n) time, O(k) space.

LIST COMPREHENSIONS

12. Square only even numbers

Question: Create a list containing squares of even numbers.

Solution:

```
nums = [1, 2, 3, 4, 5, 6]
squares = [x ** 2 for x in nums if x % 2 == 0]
print(squares)
# [4, 16, 36]
```

Explanation: The expression before for is the output. The if clause filters which values enter the output.

Complexity: O(n) time, O(n) output space.

13. Clean strings with a comprehension

Question: Strip whitespace and lowercase every name.

Solution:

```
names = [" Alice ", "BOB", " Charlie "]
cleaned = [name.strip().lower() for name in names]
print(cleaned)
# ['alice', 'bob', 'charlie']
```

Explanation: This is a practical data-cleaning pattern used constantly with real datasets.

Complexity: O(total characters) time.

14. Flatten a list of lists

Question: Convert [[1,2], [3,4], [5]] into [1,2,3,4,5].

Solution:

```
matrix = [[1, 2], [3, 4], [5]]
flat = [value for row in matrix for value in row]
print(flat)
# [1, 2, 3, 4, 5]
```

Explanation: Read it as: for each row, then for each value in that row, output the value.

Complexity: O(n) time and O(n) output space.

15. Filter and transform in one comprehension

Question: From a list of strings, return lowercase versions of words longer than 4 characters.

Solution:

```
words = ["AI", "Python", "data", "science", "ML"]
result = [w.lower() for w in words if len(w) > 4]
print(result)
```

```
# ['python', 'science']
```

Explanation: This combines filtering and transformation. Use comprehensions when the expression remains readable; avoid making them overly complicated.

Complexity: $O(n)$ time plus string-processing cost.

STRINGS

16. Reverse a string

Question: Return the reverse of a string.

Solution:

```
def reverse_string(s):  
    return s[::-1]  
  
print(reverse_string("python"))  
# nohtyp
```

Explanation: Python slicing with a step of -1 is the idiomatic solution. In interviews, also understand the two-pointer idea.

Complexity: $O(n)$ time, $O(n)$ output.

17. Palindrome check

Question: Ignore case, spaces, and punctuation, then determine whether a string is a palindrome.

Solution:

```
def is_palindrome(s):  
    cleaned = ""  
    for ch in s.lower():  
        if ch.isalnum():  
            cleaned += ch  
    return cleaned == cleaned[::-1]  
  
print(is_palindrome("A man, a plan, a canal: Panama"))  
# True
```

Explanation: Normalize the input and compare it with its reverse. A two-pointer implementation can reduce extra memory.

Complexity: $O(n)$ time, $O(n)$ space for this implementation.

18. Count vowels and consonants

Question: Count alphabetic vowels and consonants, ignoring digits and punctuation.

Solution:

```
def count_vowels_consonants(s):  
    vowels = set("aeiou")  
    v = c = 0  
  
    for ch in s.lower():  
        if ch.isalpha():  
            if ch in vowels:
```

```

v += 1
else:
c += 1
return v, c

print(count_vowels_consonants("Hello World! 123"))
# (3, 7)

```

Explanation: isalpha() filters out non-letters. A set makes vowel membership easy.

Complexity: O(n) time, O(1) auxiliary space.

19. Check if two strings are anagrams

Question: Return True if two strings have the same characters and frequencies, ignoring case and spaces.

Solution:

```

def are_anagrams(a, b):
a = [ch.lower() for ch in a if ch.isalnum()]
b = [ch.lower() for ch in b if ch.isalnum()]
return sorted(a) == sorted(b)

print(are_anagrams("listen", "silent"))
# True

```

Explanation: Sorting gives a simple solution. A Counter/dictionary frequency solution can achieve linear expected time.

Complexity: O(n log n) time with sorting.

20. Compress consecutive characters

Question: Convert 'aaabbcaaaa' into 'a3b2c1a3'.

Solution:

```

def compress(s):
if not s:
return ""

result = []
count = 1

for i in range(1, len(s)):
if s[i] == s[i - 1]:
count += 1
else:
result.append(s[i - 1] + str(count))
count = 1

result.append(s[-1] + str(count))
return "".join(result)

print(compress("aaabbcaaaa"))
# a3b2c1a3

```

Explanation: Track the length of the current run. When the character changes, save the previous run and reset the counter.

Complexity: O(n) time, O(n) output space.

ARRAYS / TWO POINTERS / SLIDING WINDOW

21. Remove duplicates from sorted array in place

Question: Given a sorted array, keep one copy of each value and return the new length.

Solution:

```
def remove_duplicates_sorted(nums):
    if not nums:
        return 0

    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[read - 1]:
            nums[write] = nums[read]
            write += 1
    return write

nums = [1, 1, 2, 2, 3, 4, 4]
n = remove_duplicates_sorted(nums)
print(nums[:n])
# [1, 2, 3, 4]
```

Explanation: Because the input is sorted, duplicates are adjacent. Read scans; write stores the next unique value.

Complexity: $O(n)$ time, $O(1)$ extra space.

22. Two Sum in sorted array

Question: Given a sorted list, return indices of two values adding to target.

Solution:

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1
    return []

print(two_sum_sorted([1, 2, 4, 7, 11], 9))
# [1, 3]
```

Explanation: If the sum is too small, move left up. If too large, move right down. Sorted order makes those decisions safe.

Complexity: $O(n)$ time, $O(1)$ space.

23. Maximum sum subarray of size k

Question: Find the maximum sum of any contiguous subarray of exactly k elements.

Solution:

```
def max_sum_k(nums, k):
    if k <= 0 or k > len(nums):
        return None

    window = sum(nums[:k])
    best = window

    for right in range(k, len(nums)):
        window += nums[right]
        window -= nums[right - k]
        best = max(best, window)

    return best

print(max_sum_k([2, 1, 5, 1, 3, 2], 3))
# 9
```

Explanation: Sliding window avoids recomputing every group of k values. Add the new right-side value and remove the value leaving the window.

Complexity: O(n) time, O(1) space.

24. Longest substring without repeating characters

Question: Return the length of the longest substring with all unique characters.

Solution:

```
def longest_unique_substring(s):
    seen = set()
    left = 0
    best = 0

    for right, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[left])
            left += 1
        seen.add(ch)
        best = max(best, right - left + 1)

    return best

print(longest_unique_substring("abcabcbb"))
# 3
```

Explanation: Maintain a window containing unique characters. When a duplicate arrives, move the left edge until the duplicate disappears.

Complexity: O(n) time, O(k) space.

25. Best time to buy and sell stock

Question: Choose one buy day and a later sell day to maximize profit.

Solution:

```
def max_profit(prices):
    min_price = float("inf")
    best = 0

    for price in prices:
        min_price = min(min_price, price)
        best = max(best, price - min_price)
```

```

return best

print(max_profit([7, 1, 5, 3, 6, 4]))
# 5

```

Explanation: Keep the cheapest price seen so far. Selling today gives price minus that minimum.

Complexity: O(n) time, O(1) space.

26. Product of array except self

Question: Return output[i] equal to the product of every number except nums[i], without division.

Solution:

```

def product_except_self(nums):
    n = len(nums)
    result = [1] * n

    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]

    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result

print(product_except_self([1, 2, 3, 4]))
# [24, 12, 8, 6]

```

Explanation: First pass stores products to the left; second pass multiplies by products to the right. This naturally handles zeros.

Complexity: O(n) time and O(1) extra space excluding output.

MIXED PRACTICAL / INTERVIEW

27. Find intersection of two lists

Question: Return unique values appearing in both lists.

Solution:

```

def intersection(a, b):
    return list(set(a) & set(b))

print(intersection([1, 2, 2, 3, 4], [2, 3, 3, 5]))
# [2, 3]

```

Explanation: Sets remove duplicates and provide an efficient intersection operation. If the interviewer requires preserving order, use a different approach.

Complexity: O(n + m) average time.

28. Top K frequent elements

Question: Return the k most frequent values.

Solution:

```
from collections import Counter

def top_k_frequent(nums, k):
    return [item for item, count in Counter(nums).most_common(k)]

print(top_k_frequent([1,1,1,2,2,3], 2))
# [1, 2]
```

Explanation: Counter is a standard-library tool worth knowing. If asked to implement the algorithm, discuss sorting, heap, or bucket approaches.

Complexity: Counter counting is $O(n)$; most_common typically involves ordering the frequencies.

29. Validate parentheses

Question: Check whether (), {}, and [] are correctly matched and nested.

Solution:

```
def valid_parentheses(s):
    stack = []
    pairs = {"(": ")", "[": "]", "{": "}"

    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False

    return not stack

print(valid_parentheses("({[]})"))
# True
print(valid_parentheses("([)])"))
# False
```

Explanation: A stack works because the most recently opened bracket must be closed first.

Complexity: $O(n)$ time, $O(n)$ space.

30. Word frequency from text

Question: Normalize a paragraph and return word frequencies.

Solution:

```
import re
from collections import Counter

def word_frequency(text):
    words = re.findall(r"[a-zA-Z]+", text.lower())
    return Counter(words)

text = "Python is great. Python is useful!"
print(word_frequency(text))
# Counter({'python': 2, 'is': 2, 'great': 1, 'useful': 1})
```

Explanation: This is directly relevant to data science and NLP: normalize, tokenize, count. The same pattern applies to logs and documents.

Complexity: $O(n)$ relative to text size, plus $O(k)$ space for k unique words.

What you should be able to do after these 30

- Write clean Python functions without relying on an IDE to autocomplete every line.
- Know when to use a list, set, or dictionary.
- Recognize frequency-counting and hashing problems.
- Recognize two-pointer and sliding-window patterns.
- Use `enumerate()`, `zip()`, `dict.get()`, `Counter`, `defaultdict`, set operations, and comprehensions naturally.
- Explain basic time and space complexity.
- Handle edge cases such as empty input, duplicates, negative numbers, and `k` larger than the input size.

Suggested order

Do Problems 1–11 first. Then 12–20. Finally do 21–30. For a fresher Data Science / ML / GenAI interview, these patterns are more useful than trying to grind hundreds of random LeetCode questions.